



UNIVERSITÀ DEGLI STUDI DI MILANO - BICOCCA

Scuola di Scienze

Dipartimento di Informatica, Sistemistica e Comunicazione

Corso di laurea Magistrale in Informatica

Architettura Dati

Report di progetto - Use Case 3

Messa Alessandro - 903321

Ronchi Matteo - 899659

Tentori Francesca - 899835

Anno Accademico 2025 - 2026

Indice

1	Introduzione	3
1.1	Scelta dello Use Case	3
1.2	Obiettivo del progetto	3
1.3	Caratteristiche del dataset	3
2	Analisi dei Dati e Addestramento dei modelli baseline	5
2.1	Exploratory Data Analysis	5
2.1.1	Analisi Univariata	5
2.1.2	Analisi Multivariata	6
2.2	Preprocessing del Dataset	8
2.2.1	Feature Dropping	8
2.2.2	Splitting Stratificato e Standardizzazione	8
2.3	Addestramento dei Modelli e Validazione Baseline	8
2.3.1	Scelta dei Modelli e Configurazione dei Parametri	8
2.3.2	Metriche di Valutazione nel Contesto Phishing	9
2.3.3	Risultati Baseline e Discussione	10
3	Sporcamento dei Dati	11
3.1	Missing data	11
3.2	Feature Occlusion	14
3.3	Targeted Feature Noise	18
3.4	Mimicry Attack	22
4	Conclusioni	25
4.1	Comparazione tra i modelli	25
4.2	Efficacia metodi di sporcamento	25
4.3	Limiti del progetto e sviluppi futuri	25
4.3.1	Specificità del dataset	25
4.3.2	Indipendenza tra le feature e vincoli strutturali	26
4.3.3	T student nel Repeated Hold-Out	26

Capitolo 1

Introduzione

NOTA RIGUARDANTE USO AI

Nello svolgimento del seguente progetto è stata utilizzata l'Intelligenza Artificiale come supporto nello scrivere grafici e parte del codice, in particolare per il Reapet Hold Out Validation usato nella fase di sporcamento. È stata anche utilizzata come correttore nella scrittura del report.

1.1 Scelta dello Use Case

Tra i quattro Use Case proposti durante il corso, è stato scelto quello relativo al **Data Management for Machine Learning**. Questa branca dell'architettura dati studia come la qualità, l'integrità e la corretta gestione dei dati di input influenzino in modo critico la robustezza e l'affidabilità dei modelli predittivi, e come la progressiva introduzione di errori, rumore o incompletezza nei dati comprometta le loro prestazioni.

La scelta è motivata dal forte interesse verso le tematiche di sicurezza informatica e dalla volontà di approfondire in che misura i modelli di Machine Learning mantengano le proprie prestazioni quando i dati in ingresso sono imperfetti o parzialmente compromessi, una condizione molto più vicina agli scenari reali di applicazione.

1.2 Obiettivo del progetto

Il presente progetto si pone l'obiettivo di valutare la robustezza di modelli di Machine Learning in scenari operativi realistici, caratterizzati da incertezza e alterazione dei dati. Tali imperfezioni possono derivare sia da omissioni e rumore fisiologico in fase di acquisizione delle metriche web, sia da attacchi di evasione mirati. In questo secondo scenario, viene simulata l'azione strategica di un attaccante che manipola intenzionalmente le caratteristiche strutturali di un sito di phishing, al fine di mimetizzarlo all'interno del traffico legittimo ed eludere i controlli di sicurezza.

1.3 Caratteristiche del dataset

Il progetto si colloca nel dominio del Machine Learning supervisionato e affronta un problema di classificazione nel contesto della sicurezza informatica, in particolare per l'identificazione di siti di phishing. È stato utilizzato il dataset **PhiUSIIL Phishing URL**, disponibile presso l'UCI Machine Learning Repository¹.

La scelta di questo dataset è stata fortemente motivata dal nostro interesse per il dominio della *Cyber-security*, affiancato da due criteri tecnici fondamentali: la dimensione campionaria e la coerenza con l'obiettivo del progetto. Nello specifico, il volume di dati a disposizione risulta sufficientemente ampio da permettere un'osservazione robusta e statisticamente rilevante dell'effetto che le tecniche di sporcamento

¹Il dataset è disponibile al seguente link: <https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset>.

hanno sulle prestazioni dei modelli.

Il dataset originale contiene 54 feature più una variabile di target, con un totale di 235.795 istanze. La variabile di target, chiamata **label**, è binaria e assegna il valore 0 se un sito è di phishing oppure 1 se è legittimo.

Capitolo 2

Analisi dei Dati e Addestramento dei modelli baseline

2.1 Exploratory Data Analysis

In questa sezione viene mostrata l'*Exploratory Data Analysis (EDA)* condotta in questo progetto prima di procedere con l'addestramento e l'applicazione degli algoritmi predittivi. L'EDA consente di determinare il modo migliore per manipolare le fonti di dati al fine di ottenere le risposte desiderate, facilitando la scoperta dei pattern, l'individuazione di anomalie, il test o la verifica di un'ipotesi¹. Al fine di garantire il rigore scientifico ed evitare ambiguità nell'interpretazione, l'analisi esplorativa viene suddivisa rigorosamente in due sotto fasi.

2.1.1 Analisi Univariata

- **Consistenza del Dataset:** L'ispezione preliminare evidenzia la corposità della base dati iniziale, la quale si compone di oltre 200.000 istanze e 54 feature. Tale consistenza quantitativa configura una solida base statistica, riducendo significativamente il rischio di overfitting legato a fluttuazioni campionarie parziali.
- **Ispezione dei Tipi di Dato e Integrità:** Mediante l'invocazione della funzione `df.info()`, sono state identificate quattro variabili categoriche. La verifica dell'integrità strutturale rileva l'assenza totale di valori mancanti (*NaN*), escludendo la necessità di ricorrere a tecniche di gestione come l'imputazione.
- **Bilanciamento della Classe Target:** Lo studio distributivo della variabile target `label`, mostrato nella figura 2.1 rileva un leggero sbilanciamento delle classi nel dataset originale: i siti web legittimi si attestano al 57.2%, mentre i siti associati ad attacchi di phishing costituiscono il 42.8%.

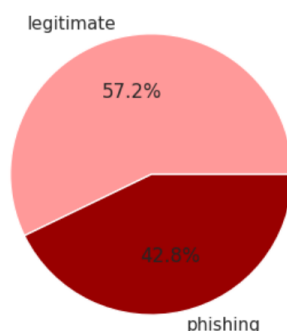


Figura 2.1: Distribuzione della classe Target nel dataset

¹<https://www.ibm.com/think/topics/exploratory-data-analysis>

- **Statistiche Descrittive delle Feature Numeriche:** L'analisi quantitativa delle statistiche descrittive mostra:

- **Eterogeneità degli Ordini di Grandezza:** Si osserva una coesistenza di feature limitate in intervalli microscopici, come URLCharProb ($\mu \approx 0.05$, $\max = 0.09$) o le variabili strettamente binarie come IsHTTPS e IsDomainIP ($[0, 1]$), con feature macroscopiche legate alla volumetria del codice sorgente.
- **Presenza di Outlier Estremi e Asimmetria:** La variabile LargestLineLength registra un valore medio di 12.789,53 a fronte di una deviazione standard imponente ($\sigma \approx 152.201,1$) e un valore massimo assoluto che tocca $1,39 \times 10^7$. Dinamiche analoghe si riscontrano in LineOfCode ($\max = 442.666$) e NoOfCSS ($\max = 35.820$).

Dal punto di vista modellistico, questi risultati indicano la necessità di una standardizzazione (*Z-score normalization*) per i modelli a transizione continua come il Neural Network.

2.1.2 Analisi Multivariata

Nel contesto della EDA multivariata, si è scelto di implementare e confrontare sistematicamente la matrice di correlazione di Pearson e la matrice di Spearman, mostrate nella Figura 2.2. Questa duplice analisi si rende necessaria per superare i limiti intrinseci di un approccio esclusivamente lineare.

Il coefficiente di Pearson valuta esclusivamente le relazioni lineari, di conseguenza presenta due principali punti ciechi nel nostro scenario:

- **Sensibilità agli outlier:** Essendo basato sugli scostamenti quadratici, basta un singolo valore estremo per alterare significativamente il coefficiente, mostrando relazioni inesistenti o mascherando legami reali.
- **Assunzione di linearità:** Molte feature del dataset sono conteggi discreti (es. NoOfImage, LineOfCode, NoOfJS). Queste variabili spesso non si muovono lungo una linea retta perfetta ma seguono un andamento monotono non lineare, scenario in cui Pearson fallisce.

Al contrario, il coefficiente di Spearman calcola la correlazione basandosi sui *ranghi*, ovvero la posizione ordinata dei dati e non il loro valore assoluto. Questa caratteristica lo rende matematicamente robusto agli outlier ed estremamente efficace nel catturare relazioni monotoniche anche non lineari.

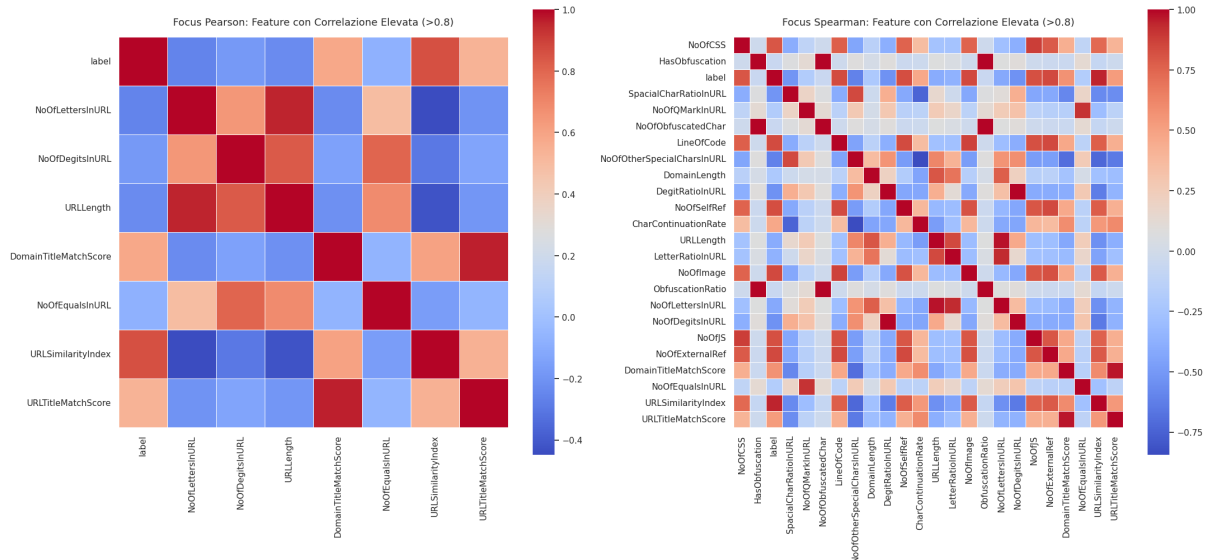


Figura 2.2: Confronto matrici di correlazione: Pearson a sinistra, Spearman a destra

Discussione dei Risultati

Pearson e Spearman hanno dato risultati molto diversi: il primo ha trovato solo 5 coppie di feature correlate sopra la soglia di 0.8, il secondo 28. Questa soglia è quella convenzionalmente usata in letteratura per indicare correlazione forte e ridondanza informativa [1], ed è un compromesso ottimale: più bassa rischierebbe di escludere relazioni moderate ma informative, più alta non intercetterebbe multicollinearità dannosa per la stima dei parametri e l'overfitting. Nel phishing, dove molte feature strutturali covariano naturalmente, questa soglia permette di isolare i blocchi ridondanti senza eliminare varianza utile ai modelli.

I risultati mostrano la presenza di forti correlazioni strutturali tra alcune feature:

- **Con Pearson:** Le feature `DomainTitleMatchScore` e `URLTitleMatchScore` mostrano una correlazione lineare quasi perfetta pari a 0.96, indicando una ridondanza diretta nel modo in cui i punteggi di corrispondenza del titolo vengono calcolati rispetto al dominio e all'intero URL. Al contempo, la lunghezza complessiva dell'URL (`URLLength`) risulta fortemente legata al conteggio assoluto dei suoi componenti principali, in particolare `NoOfLettersInURL` e `NoOfDgitsInURL`, evidenziando come la dimensionalità lineare dipenda in gran parte da queste singole metriche quantitative.
- **Con Spearman:** Si osserva una correlazione geometrica (pari o prossima a 1.00) nel cluster relativo alle tecniche di elusione (`NoOfObfuscatedChar`, `ObfuscationRatio` e `HasObfuscation`). Questo comportamento è giustificato da un vincolo deterministico: la presenza stessa di offuscamento attiva la variabile booleana e ne stabilisce matematicamente il rapporto rispetto alla lunghezza. Una dinamica analoga, legata al legame intrinseco tra conteggi assoluti e relative proporzioni strutturali, si riscontra nelle coppie legate alla composizione dell'URL, come `NoOfDgitsInURL` e `DgitRatioInURL` (0.99) o `NoOfLettersInURL` e `LetterRatioInURL` (0.93). Risulta particolarmente significativa la correlazione tra `NoOfEqualsInURL` e `NoOfQMarkInURL` (0.91), riflesso della struttura sintattica delle *query string* nei link ingannevoli. Emergono forti coppie di occorrenze tra la dimensione del codice e gli elementi multimediali (`LineOfCode` e `NoOfImage` a 0.86) e tra i collegamenti interni ed esterni (`NoOfSelfRef` e `NoOfExternalRef` a 0.84). Infine, l'indice rileva un'elevata correlazione tra `NoOfCSS` e `NoOfJS` (0.88). È fondamentale evidenziare che un approccio puramente automatico di *feature selection* che preveda l'eliminazione di una delle due variabili risulterebbe metodologicamente scorretto. Sebbene dal punto di vista statistico esse si muovano in modo sincrono esse mappano pilastri semantici completamente distinti; un sito malevolo può stravolgere gli script per evadere i controlli, mantenendo però intatta la struttura CSS per imitare fedelmente il brand target.

Per quanto riguarda la relazione con la variabile target (`label`):

- **Con Pearson:** La variabile `label` mostrava un legame forte esclusivamente con la feature `URLSimilarityIndex` (0.86), facendo apparire le restanti metriche come statisticamente ininfluenti o slegate dal fenomeno del phishing.
- **Con Spearman:** Si è assistito a un'espansione dei coefficienti, svelando una correlazione superiore a 0.80 tra il target e numerose feature strutturali del codice e della pagina, tra cui `NoOfExternalRef` (0.84), `NoOfImage` (0.84), `LineOfCode` (0.84), `NoOfSelfRef` (0.82), `NoOfJS` (0.82) e `NoOfCSS` (0.80).

2.2 Preprocessing del Dataset

Questa sezione descrive le operazioni ingegneristiche e le trasformazioni vettoriali applicate al dataset al fine di ottimizzare i vettori di input prima della fase di modeling.

2.2.1 Feature Dropping

Dalle conclusioni emerse dall'analisi multivariata, si è proceduto alla rimozione mirata di 13 colonne, riducendo lo spazio delle feature da 54 a 41 variabili pulite. Questa operazione risponde a tre fondamentali requisiti metodologici: la gestione della cardinalità estrema, l'evitamento del *data leakage* e la risoluzione della multicollinearità perfetta rilevata dalle matrici di Pearson e Spearman.

In primo luogo, le variabili categoriche testuali `Domain`, `URL`, `TLD` e `Title` sono state rimosse a causa della loro natura non strutturata e della presenza di valori quasi interamente univoci per ogni record del dataset. Un eventuale tentativo di codifica di questi campi avrebbe causato un'esplosione incontrollata della dimensionalità, dilatando i tempi di addestramento e spingendo i modelli di Machine Learning verso l'overfitting.

In secondo luogo, si è scelto di scartare le feature `URLSimilarityIndex` e `TLDLegitimateProb`, entrambe estremamente correlate con la `label` e propense a creare *Data Leakage*. Si tratta di metriche complesse e derivate, calcolate a monte dagli autori del paper originale tramite euristiche proprie.

Per le restanti feature, si è scelto di eliminare i conteggi assoluti per preservare esclusivamente le rispettive metriche relative (i *ratio*), che risultano molto più informative. Nello specifico, `URLTitleMatchScore` è stato rimosso per via della sua correlazione quasi perfetta (0.96) con `DomainTitleMatchScore`. Allo stesso modo, le feature `NoOfObfuscatedChar` e `HasObfuscation` sono state eliminate in favore di `ObfuscationRatio` (con cui mostravano una correlazione di 1.00). Con la medesima logica, la lunghezza complessiva `URLLength` e i conteggi atomici `NoOfDigitsInURL`, `NoOfLettersInURL` e `NoOfOtherSpecialCharsInURL` sono stati rimossi.

2.2.2 Splitting Stratificato e Standardizzazione

Il dataset viene suddiviso rigorosamente in Training Set (80%) e Test Set (20%). Viene applicata la stratificazione rispetto al target (`stratify=y_model`), garantendo che l'esatta proporzione originaria (57.2% vs 42.8%) sia preservata matematicamente in entrambi i sottoinsiemi.

Successivamente i dati sono stati standardizzati per impedire fenomeni di *Gradient Explosion* nelle reti neurali, è stato istanziato un oggetto `StandardScaler`, configurando la corretta prassi metodologica: il fit viene eseguito esclusivamente sui dati di train (`fit_transform`) e applicato per trascinamento sul test set (`transform`), azzerando il rischio di *Data Leakage*.

2.3 Addestramento dei Modelli e Validazione Baseline

In questa sezione vengono mostrati i modelli scelti e le metriche di valutazione con i relativi risultati.

2.3.1 Scelta dei Modelli e Configurazione dei Parametri

Per affrontare il problema della classificazione dei siti web di phishing, si è scelto di confrontare un modello di Decision Tree e un modello di Neural Network, allo scopo di contrapporre un'architettura basata su regole rigide e trasparenti a un modello distribuito ad altissime prestazioni.

Decision Tree Classifier

Il primo modello selezionato è il Decision Tree, un modello di classificazione basato su una struttura gerarchica di decisioni, in cui lo spazio delle istanze viene suddiviso iterativamente tramite regole basate sugli attributi. Esso seleziona implicitamente le variabili più informative ad ogni split, ignorando quelle non rilevanti.

Il modello è stato istanziato utilizzando la libreria `scikit-learn` con i seguenti iperparametri chiave:

- **max_depth = 7:** limitare la profondità massima dell'albero a 7 livelli serve a prevenire l'overfitting sui dati di training, costringendo il modello a generalizzare le regole senza creare foglie che contengano singoli campioni isolati.
- **min_samples_split = 2:** il numero minimo di campioni richiesti per dividere un nodo interno.

- **class_weight = 'balanced'**: sebbene il dataset non presenti uno sbilanciamento estremo, l'impostazione di pesi bilanciati assicura che il modello non sviluppi bias verso la classe maggioritaria, penalizzando proporzionalmente in modo maggiore gli errori commessi sulla classe minoritaria.

Neural Network (NN)

Il secondo modello implementato è il Neural Network, creato tramite la libreria **TensorFlow/Keras**. Essendo un modello sensibile alla scala delle feature in ingresso, i dati numerici sono stati preventivamente sottoposti a standardizzazione.

L'architettura è stata progettata per essere efficiente e limitare la complessità computazionale:

- **Topologia dei layer**: Il modello presenta un layer di input seguito da due *Hidden Dense Layer* rispettivamente da 32 e 16 neuroni, entrambi dotati di funzione di attivazione **ReLU** per catturare relazioni non lineari. Il layer di output è composto da un singolo neurone con funzione di attivazione **Sigmoid**, che restituisce una probabilità continua compresa tra 0 e 1.
- **Ottimizzazione e Loss**: La rete è stata compilata utilizzando l'ottimizzatore stocastico **Adam** e la funzione di perdita **binary_crossentropy**.
- **Addestramento e Early Stopping**: Il training è stato impostato per un massimo di 30 epoche con una **batch_size** di 512 per velocizzare l'apprendimento su un dataset di grandi dimensioni. È stata inoltre applicata una *callback* di **EarlyStopping**, con **patience=2**, monitorando il tasso di true negative di validazione. Questo meccanismo permette di interrompere l'addestramento non appena il modello smette di migliorare sui dati non visti.

2.3.2 Metriche di Valutazione nel Contesto Phishing

L'accuratezza globale non è sufficiente per valutare un sistema di rilevamento phishing, sia per il lieve sbilanciamento delle classi, sia perché un falso positivo e un falso negativo hanno conseguenze molto diverse. I modelli sono stati quindi valutati sul Test Set, con 47.159 istanze, con un set più ampio di metriche.

Nei casi di sicurezza come il nostro, la classe di reale interesse è la classe 0, e non la media aggregata (*macro avg*) tra le due classi, che potrebbe mascherare un comportamento non ottimale del modello proprio sulla classe più critica.

- **Accuracy**: Misura la percentuale di previsioni globalmente corrette.
- **Precision**: Misura la proporzione di siti realmente di phishing/legittimi tra tutti quelli etichettati come tali dal modello.
- **Recall**: Misura la proporzione di veri siti di phishing che il modello è riuscito a intercettare. Nel caso della classe 0, una bassa recall si traduce in conseguenti rischi di furto di credenziali.
- **F1-Score**: Media armonica tra Precision e Recall. Viene utilizzata come metrica di controllo, per verificare che un eventuale miglioramento della Recall non avvenga a scapito di un numero eccessivo di falsi allarmi.

In sintesi, sebbene tutte le metriche vengano riportate per completezza nei risultati sperimentali, il confronto tra Decision Tree e Neural Network sarà guidato principalmente dalla Recall calcolata sulla classe 0 in quanto in questo dominio il costo di un Falso Positivo è superiore a quello di un Falso Negativo.

2.3.3 Risultati Baseline e Discussione

Le previsioni ottenute dai modelli sul Test Set originale, quello privo di tecniche di sporcamento dei dati, sono riassunte nella Tabelle 2.1 e 2.2.

Classe / Metrica	Precision	Recall	F1-Score	Support
0 (phishing)	0.9962	0.9947	0.9955	20189
1 (legittimo)	0.9961	0.9971	0.9966	26970
Accuracy			0.9961	47159
Macro Avg	0.9961	0.9959	0.9960	47159
Weighted Avg	0.9961	0.9961	0.9961	47159

Tabella 2.1: Classification Report del modello Baseline del Decision Tree

Classe / Metrica	Precision	Recall	F1-Score	Support
0 (phishing)	0.9998	0.9995	0.9996	20189
1 (legittimo)	0.9996	0.9998	0.9997	26970
Accuracy			0.9997	47159
Macro Avg	0.9997	0.9997	0.9997	47159
Weighted Avg	0.9997	0.9997	0.9997	47159

Tabella 2.2: Classification Report del modello Baseline della Neural Network

Sebbene entrambi i modelli ottengono prestazioni molto alte, in un contesto di sicurezza informatica il fattore discriminante è la Recall della classe phishing. In base a questo, il Decision Tree ottiene una Recall pari a 0.9947, fallendo nell'identificazione di circa 106 siti di phishing su 20.189. La Neural Network, invece, raggiunge un valore di 0.9996, riducendo il numero di Falsi Positivi a 13. Quest'ultima quindi garantisce un margine di sicurezza significativo.

Capitolo 3

Sporcamento dei Dati

Per valutare la robustezza del Decision Tree e della Neural Network, il dataset viene sottoposto a quattro differenti tecniche di sporcamento, misurando il progressivo degrado delle prestazioni rispetto alla baseline. Al fine di garantire il rigore statistico, per ciascuna tecnica di sporcamento viene adottato un protocollo di validazione standardizzato basato su una procedura di **Repeated Hold-Out Validation**. Il dataset viene ripartito $N = 10$ volte in sottoinsiemi di addestramento (80%) e di test (20%) tramite split stratificati indipendenti, preservando matematicamente la proporzione originaria delle classi (57.2% siti legittimi e 42.8% siti di phishing).

Per ogni iterazione della procedura, i modelli vengono riaddestrati da zero sul set informativo di train corrente. Al fine di evitare Data Leakage, tutti i parametri di preprocessing vengono calcolati esclusivamente sulla porzione di addestramento locale e applicati per trascinamento transform sul test set corrotto.

I risultati quantitativi riportati nelle analisi successive rappresentano la media delle metriche sulle $N = 10$ ripetizioni. Le barre di errore visualizzate nei grafici delineano l'intervallo di confidenza al $\pm 95\%$, calcolato mediante la distribuzione t di Student con $N - 1 = 9$ gradi di libertà.

3.1 Missing data

Descrizione

La tecnica di *Missing Data* simula l'assenza di valori nel dataset di test riproducendo il meccanismo statistico noto come **MCAR** (*Missing Completely At Random*), formalizzato nella tassonomia classica di Little e Rubin [2]. Secondo tale definizione, la matrice di missingness è stocasticamente indipendente sia dai valori osservati X_{obs} sia da quelli mancanti X_{mis} . In altri termini, la probabilità che una cella diventi mancante è identica per ogni feature e per ogni osservazione:

$$P(M \mid X_{\text{obs}}, X_{\text{mis}}) = P(M) \quad (3.1)$$

dove M è la matrice booleana indicatrice del missing rate.

Motivazione

Nello specifico contesto dell'analisi di URL, numerose feature del dataset (ad esempio **NoOfImage**, **LineOfCode**, **HasTitle**) possono essere calcolate soltanto se la pagina collegata all'URL viene effettivamente scaricata e renderizzata. Questo scenario fallisce frequentemente a causa di timeout di rete, contenuti generati dinamicamente via JavaScript, meccanismi anti-bot o semplice indisponibilità del sito. Valutare la robustezza dei modelli in presenza del progressivo aumento di dati mancanti permette di stimare le prestazioni attese in un contesto operativo realistico, invece che nel solo scenario idealizzato di un dataset completo.

Funzionamento

Per ogni cella $x_{i,j}$ del test set X_{test} si genera un valore pseudocasuale attraverso una distribuzione uniforme $u_{i,j} \sim \mathcal{U}(0, 1)$. Dato un parametro p_{miss} (**missing_rate**):

$$x_{i,j} = \begin{cases} \text{NaN} & \text{se } u_{i,j} < p_{\text{miss}} \\ x_{i,j} & \text{altrimenti} \end{cases} \quad (3.2)$$

Il risultato è che, in media, una frazione p_{miss} delle celle totali del dataset risulta mancante, distribuita in modo uniforme su tutte le celle.

Per ripristinare la dimensionalità del dataset richiesta dai classificatori, valori mancanti vengono poi ricostruiti tramite imputazione, sostituendo ciascun NaN con la mediana della rispettiva feature calcolata esclusivamente sul training set.

Applicazione nel progetto

Valutiamo cinque livelli crescenti di `missing_rate` $p_{\text{miss}} \in \{0\%, 5\%, 10\%, 20\%, 40\%\}$, applicati indipendentemente a tutte le feature del test set X_{test} . I valori mancanti generati vengono imputati mediante un `SimpleImputer` con `strategy='median'`, addestrato esclusivamente su X_{train} per evitare data leakage. Il dataset ricostruito viene poi utilizzato per calcolare le predizioni sia del Decision Tree sia della rete neurale, confrontando Accuracy, Precision, Recall ed F1-score al variare di p_{miss} .

Risultati ottenuti

I risultati empirici illustrati nelle Figure 3.1 e 3.2 evidenziano il progressivo degradamento delle performance all'aumentare del tasso di valori mancanti.

Come delineato nella Sezione 2.3.2, focalizzando l'analisi sulla *Recall* della classe di phishing, emerge una netta divergenza strutturale tra le due architetture. La **Neural Network** si dimostra decisamente più robusta: anche a fronte di un *missing rate* estremo del 40%, la *Recall* flette in modo contenuto, passando da 1.000 a 0.920. Questa resilienza deriva dalla capacità della rete di assorbire e distribuire il rumore sull'intera topologia dei pesi.

Al contrario, il **Decision Tree** subisce un collasso lineare e severo, crollando da una *baseline* di 0.995 a 0.716. Operando attraverso rigidi criteri di partizionamento ortogonale (*split*), l'albero viene profondamente destabilizzato dalla perdita dei valori originari sui nodi decisionali chiave.

Infine, l'analisi degli intervalli di confidenza rivela un'ulteriore criticità: all'aumentare del tasso di rumore, la varianza campionaria del Decision Tree si dilata sensibilmente rispetto a quella della Neural Network, che rimane quasi impercettibile. Ciò indica che il degrado dell'albero non è soltanto più marcato in termini assoluti, ma diviene anche fortemente instabile e imprevedibile al variare della specifica partizione di train/test.

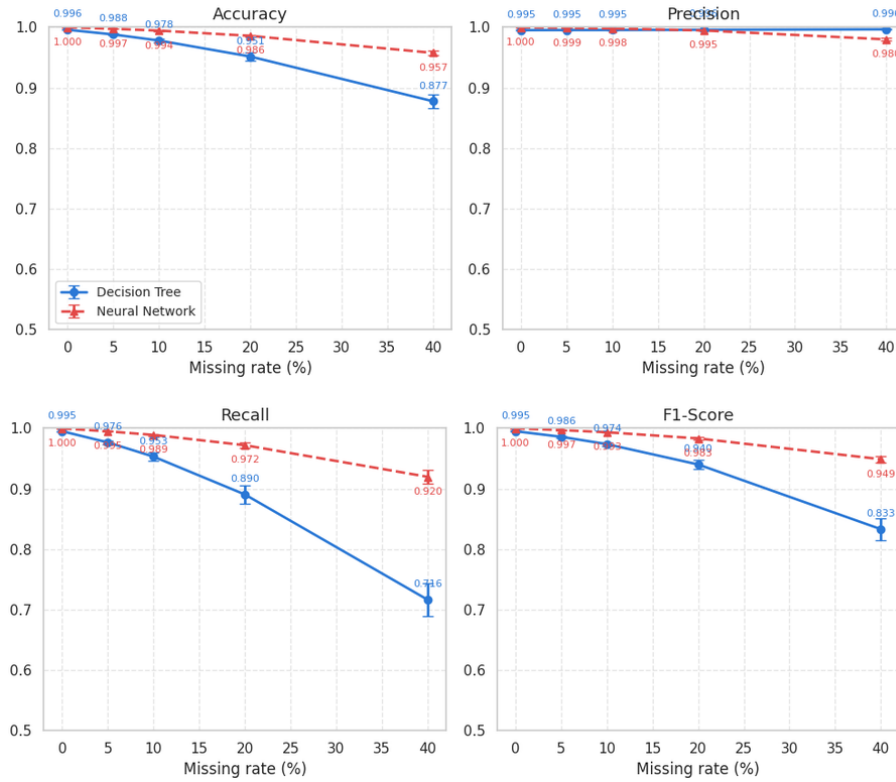


Figura 3.1: Degradazione performance MCAR - classe 0

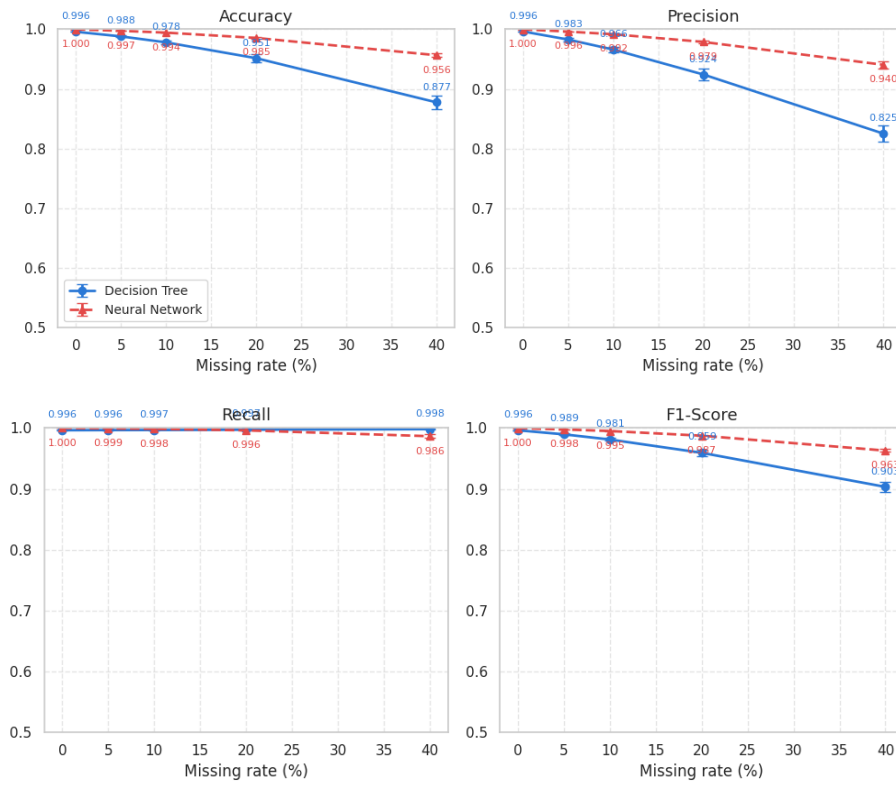


Figura 3.2: Degradazione performance MCAR - classe 1

3.2 Feature Occlusion

Descrizione

Nell'ambito dell'analisi di robustezza e spiegabilità dei modelli predittivi, lo studio di *Feature Occlusion* rappresenta una metodologia volta a quantificare l'esatto contributo informativo strutturale che un modello assegna a ciascuna variabile di input.

A differenza di uno studio di ablazione classico (*True Feature Ablation*), in cui i classificatori vengono ri-addestrati da zero escludendo progressivamente una feature alla volta, l'approccio qui implementato preserva l'architettura dei modelli precedentemente fittati sul set informativo completo. L'intervento di oscuramento avviene esclusivamente a tempo di inferenza (*inference time*) sul set di test, iniettando un disturbo controllato che neutralizza localmente il segnale informativo della singola variabile sotto analisi, mantenendo inalterata la topologia e i pesi del modello.

Motivazione

La conseguenza dell'adozione di questa tecnica è la necessità di simulare il reale scenario operativo in cui i sistemi di rilevamento basati su Machine Learning vengono distribuiti e attaccati. Nel dominio della *Cybersecurity*, infatti, un malintenzionato che progetta una campagna di *phishing* non agisce in modo casuale, ma altera deliberatamente la struttura delle pagine web e la composizione sintattica degli URL con l'obiettivo esplicito di ingannare sia l'utente finale sia i classificatori automatici.

Tecniche comuni di offuscamento, come l'iniezione massiva di codice benigno artificiale, il bilanciamento strategico del numero di script JavaScript o la manipolazione del numero di collegamenti ipertestuali esterni, si traducono, dal punto di vista analitico, in una perturbazione mirata del vettore delle feature numeriche. Iniettare un disturbo controllato all'interno del Test Set, forzando una variabile alla volta al valore della sua media campionaria, equivale a privare improvvisamente il modello di uno specifico pilastro informativo.

Questa operazione riproduce fedelmente l'effetto di un attacco di elusione in cui l'attaccante riesce a neutralizzare l'efficacia di una feature critica per far apparire la pagina malevola come legittima.

Funzionamento

La neutralizzazione del segnale informativo avviene sostituendo, all'interno del set di test, i valori reali della feature sotto esame con la media campionaria ($\bar{x}_{\text{train}}$) calcolata sul rispettivo sottoinsieme di addestramento. Questo approccio di *imputation* proietta la variabile in un punto d'ancoraggio statisticamente neutro rispetto alla distribuzione della popolazione, prevenendo anomalie strutturali o il *crash* computazionale degli algoritmi. Il calo prestazionale per ciascuna metrica m viene quantificato come lo scostamento diretto rispetto alla *baseline* integra:

$$d_m = \text{Metrica}_{\text{baseline}} - \text{Metrica}_{\text{occluded}} \quad (3.3)$$

I valori così ottenuti vengono infine aggregati lungo le diverse iterazioni del protocollo sperimentale per ricavare la stima finale del crollo prestazionale e la relativa deviazione standard.

Applicazione nel progetto

Al fine di ottimizzare il carico computazionale, l'architettura dei cicli di valutazione è stata progettata in modo condiviso per entrambi i modelli. I classificatori vengono addestrati una sola volta per ciascuno dei 10 macro-split della validazione incrociata. Successivamente, un ciclo secondario scorre l'array delle feature applicando la maschera di occlusione (basata sul valore medio) sul set di test perturbato ed eseguendo unicamente il metodo inferenziale `predict()`, velocizzando drasticamente i tempi di esecuzione, in particolar modo per la Rete Neurale.

I risultati aggregati vengono infine proiettati su una griglia grafica simmetrica 4×2 . Ciascun sotto-grafico visualizza le barre orizzontali dei drop prestazionali completi di barre d'errore basate sulla deviazione standard del drop stesso. Al fine di evitare distorsioni visive e garantire l'integrità scientifica del report, viene applicata un'unificazione della scala degli assi per tutti i sotto-grafici, permettendo di confrontare istantaneamente l'entità dei crolli prestazionali tra le diverse metriche ed evidenziando visivamente la reale sensibilità dei classificatori.

Risultati ottenuti

L'analisi comparativa tra il *Decision Tree* (Figura 3.3) e la *Neural Network* (Figura 3.4) evidenzia risposte strutturalmente opposte alla tecnica di iniezione del rumore.

Il **Decision Tree** mostra una forte vulnerabilità gerarchica. A fronte di una baseline di 0.995, l'occlusione della singola feature **NoOfImage** causa un crollo significativo della *Recall* sul phishing di 66. Inoltre, anche le feature: *LineOfCode*, *NoOfSelfRef* e *NoOfExternalRef* ad avere un impatto significativo sulla metrica di Recall con perdite che vanno da 15 a 5 punti percentuali.

Al contrario, la **Neural Network** dimostra una resilienza distribuita. Nonostante la baseline quasi perfetta (0.999), la rimozione della feature più impattante (**LetterRatioInURL**) provoca un decremento della *Recall* limitato a soli 3 punti percentuali.

Osservando i grafici, emerge un ulteriore dettaglio critico legato agli intervalli di confidenza. Le barre del Decision Tree si allargano visibilmente, raggiungendo un delta ($val_{max} - val_{min}$) massimo di 20pp. Al contrario, gli intervalli della Neural Network rimangono estremamente ristretti con un delta massimo di 2pp. Questo dimostra visivamente che il Decision Tree non solo subisce un crollo prestazionale medio peggiore, ma diventa anche nettamente più instabile e incerto nelle sue predizioni rispetto alla forte stabilità mantenuta dalla Neural Network.

In conclusione, i risultati confermano che le variabili più che penalizzano l'albero corrispondono esattamente alle relazioni monotoniche identificate in precedenza dall'indice di Spearman.

Occlusion study — Decision Tree (Stratified 10-Repeated Hold-Out)

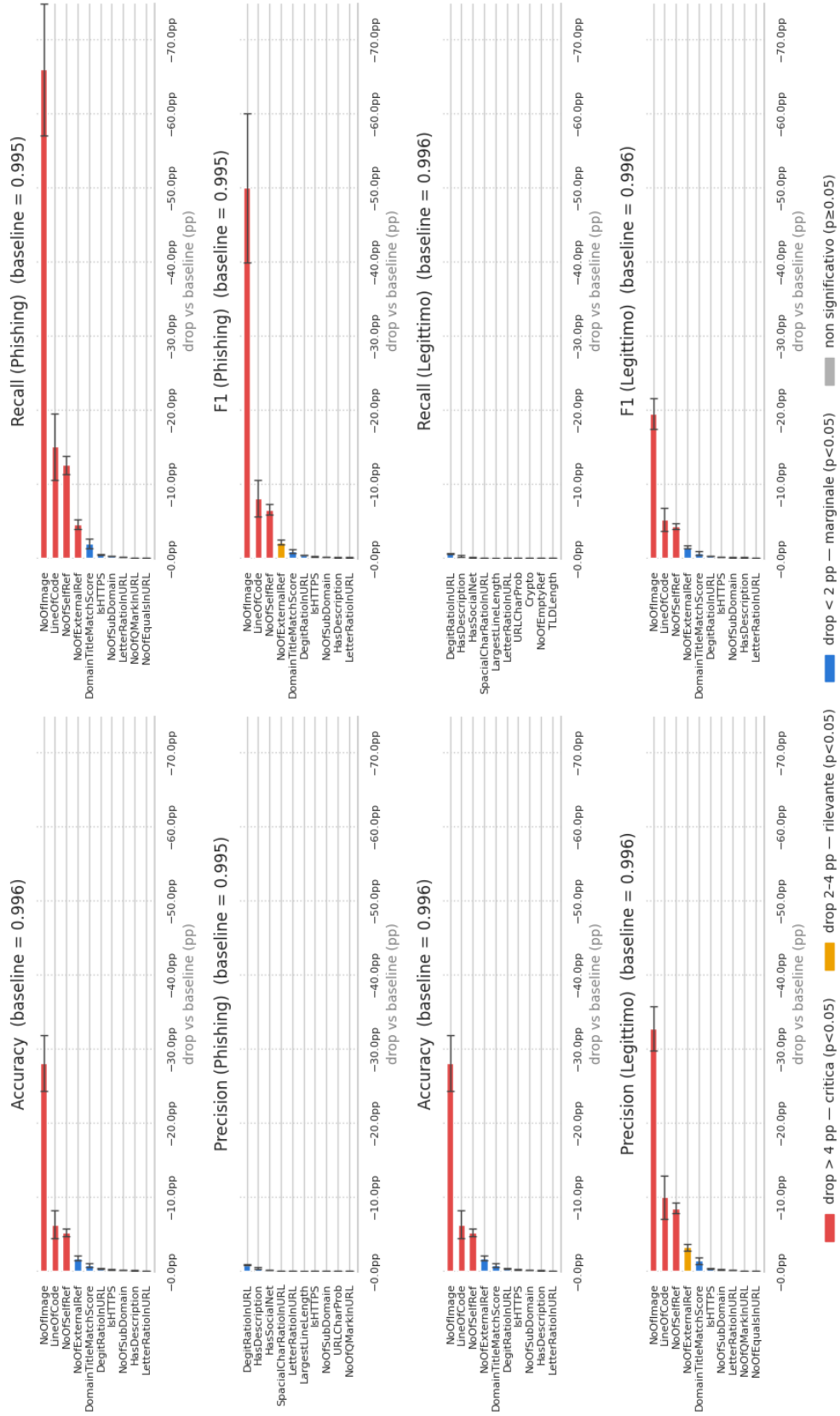


Figura 3.3: Degradazione delle performance Feature Occlusion - Decision Tree

Occlusion study — Neural Network (Stratified 10-Repeated Hold-Out)

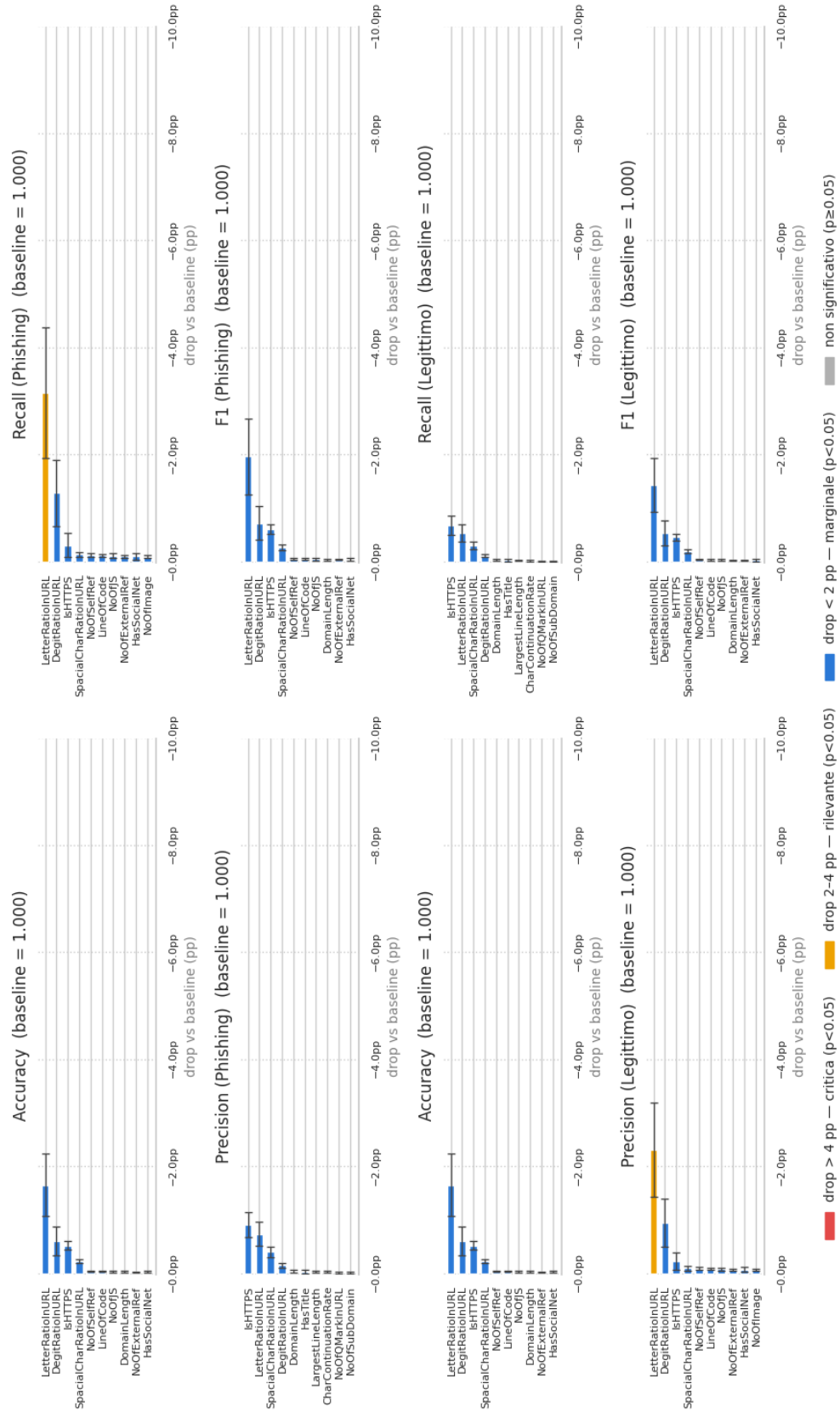


Figura 3.4: Degradazione delle performance Feature Occlusion - Neural Network

3.3 Targeted Feature Noise

Descrizione

La tecnica di *Targeted Feature Noise* consiste nell'applicare una perturbazione casuale, nel nostro caso gaussiana e poi uniforme, esclusivamente al sottoinsieme di feature identificate come critiche dall'Feature Occlusion Study (Sezione 3.2), anziché a tutte le feature del dataset indistintamente.

Motivazione

L'obiettivo di questo esperimento è valutare la robustezza del modello rispetto alle feature da cui dipende maggiormente.

Se un modello si affida maggiormente a un ristretto sottoinsieme di feature per la propria decisione, è lecito chiedersi quanto tale dipendenza sia stabile. Concentrare la perturbazione sulle sole feature rilevanti, anziché su tutte le 41 disponibili, permette di isolare l'effetto della corruzione: un eventuale calo di performance è così attribuibile in modo diretto all'importanza di quelle feature per il modello.

Il rumore è stato applicato su tutto il test set, indipendentemente dalla classe di appartenenza dei campioni. Questa scelta non vuole misurare la vulnerabilità del modello a un attacco di evasione intenzionale, ma la sua stabilità generale in presenza di corruzioni casuali sulle feature più critiche, a prescindere da un ipotetico intento malevolo.

Sono state utilizzate due distribuzioni di rumore distinte, gaussiana e uniforme. Il rumore gaussiano, concentrato attorno a variazioni piccole, testa la tenuta del modello a perturbazioni contenute e realistiche; il rumore uniforme, che assegna pari probabilità a qualunque ampiezza entro un intervallo definito, rappresenta uno scenario di corruzione più severo, includendo anche perturbazioni ampie.

Il confronto tra i due permette di capire se il modello è fragile solo in condizioni estreme o già in presenza di variazioni minime, offrendo un'indicazione più fine sulla "durezza" del confine decisionale attorno alle feature più influenti.

Funzionamento

Sia $F \subset \{1, \dots, p\}$ l'insieme degli indici delle feature critiche individuate dal Feature Occlusion (`target_features`). Per ogni feature $j \in F$ si applica una perturbazione δ_j , lasciando invariate tutte le feature $j \notin F$:

$$x'_{i,j} = \begin{cases} x_{i,j} + \delta_{i,j} & \text{se } j \in F \\ x_{i,j} & \text{se } j \notin F \end{cases} \quad (3.4)$$

dove $\delta_{i,j}$ è campionato secondo una delle due distribuzioni già utilizzate per il rumore generale:

$$\delta_{i,j} \sim \mathcal{N}(0, \sigma^2) \quad (\text{rumore gaussiano}) \quad (3.5)$$

$$\delta_{i,j} \sim \mathcal{U}(-a, a) \quad (\text{rumore uniforme}) \quad (3.6)$$

con σ e a pari al livello di rumore testato.

Applicazione nel progetto

Utilizziamo le funzioni `add_gaussian_noise` e `add_uniform_noise`, applicandole però esclusivamente alle colonne critiche individuate dal Feature Occlusion Study. Poiché il Feature Occlusion è stata condotta separatamente per i due modelli, gli insiemi di feature target differiscono: per il Decision Tree, `target_features_dt = {NoOfImage, LineOfCode, NoOfSelfRef, NoOfExternalRef}`, mentre per la Neural Network ha individuato una singola feature dominante, `target_features_nn = {LetterRatioInURL}`. Per la Neural Network, la perturbazione viene applicata sui dati grezzi (coerentemente con la scala usata per il Decision Tree) e solo successivamente il test set viene standardizzato tramite lo `scaler` adattato sul training set di quello split, in modo che i livelli di rumore restino confrontabili tra i due modelli. Testiamo entrambi i tipi di rumore sugli stessi cinque livelli di intensità $\{0, 0.5, 1, 2, 5\}$. I risultati riportati rappresentano la media sulle N ripetizioni, con intervalli di confidenza al 95%

Risultati ottenuti

L'applicazione del *Targeted Feature Noise* conferma la marcata differenza di robustezza tra le due architetture. Analizzando la *Recall* sulla classe di phishing, emerge un divario strutturale evidente in entrambi gli scenari.

Il **Decision Tree** evidenzia un collasso severo: degradando i suoi nodi decisionali primari, la *Recall* precipita da 0.995 a 0.817. Questa fragilità è confermata dall'analisi degli intervalli di confidenza, che mostrano un progressivo e marcato allargamento all'aumentare del tasso di corruzione. Le soglie rigide dell'albero si rivelano estremamente vulnerabili sia alla varianza gaussiana che a quella uniforme, rendendo il modello non solo inefficace sotto attacco, ma anche profondamente instabile e incerto nelle sue risposte. Al contrario, la **Neural Network** dimostra una resilienza eccezionale all'attacco mirato. Anche corrompendo fino al 40% dei valori delle feature chiave, la *Recall* subisce una flessione marginale (da 1.000 a 0.970). Inoltre, l'ampiezza degli intervalli di confidenza rimane quasi impercettibile lungo tutto l'asse di intensità del rumore: la rete assorbe le fluttuazioni gaussiane e uniformi distribuendole sui pesi sinaptici, garantendo una stabilità assoluta nelle predizioni indipendentemente dalla partizione dei dati.

Targeted Feature Noise — Decision Tree media $\pm$ IC 95% su 10 split

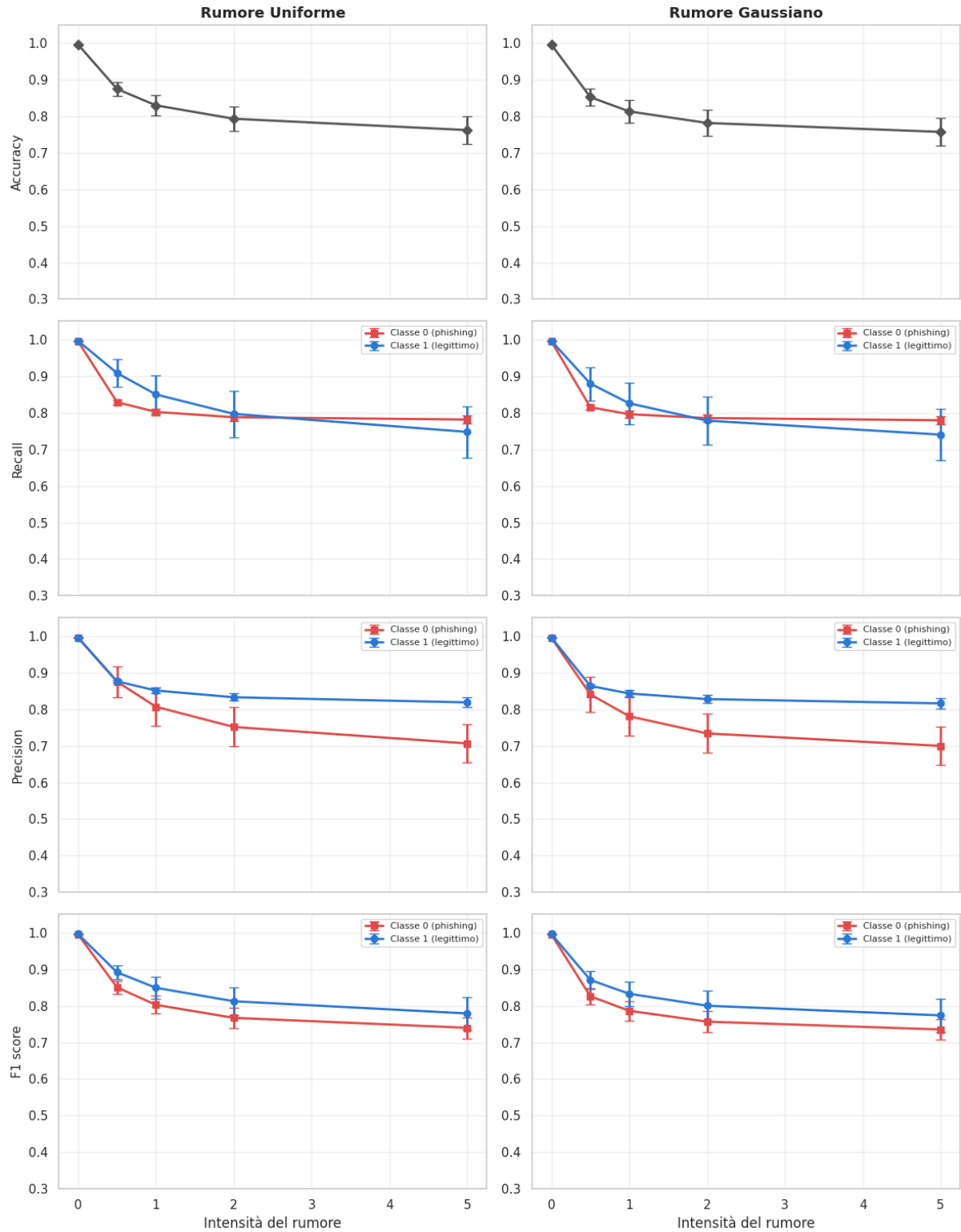


Figura 3.5: Degradazione performance Target Feature Noise - Decision Tree

Targeted Feature Noise — Neural Network media $\pm$ IC 95% su 10 split

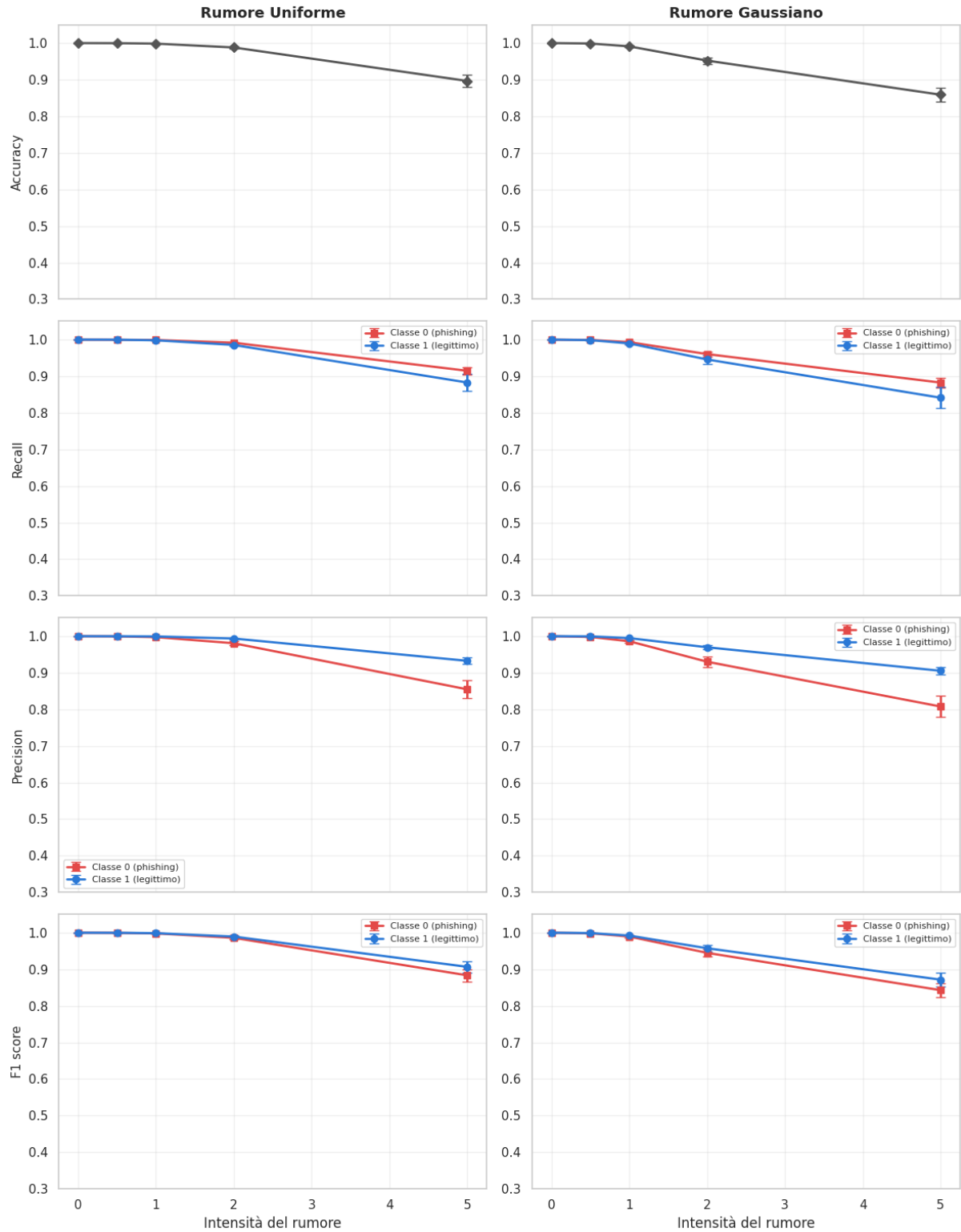


Figura 3.6: Degradazione performance Target Feature Noise - Neural Network

3.4 Mimicry Attack

Descrizione

Il Mimicry Attack costituisce una delle tecniche di elusione a tempo di inferenza più sofisticate e studiate nell'ambito dell'Adversarial Machine Learning. A differenza delle tradizionali perturbazioni stocastiche o dell'iniezione di rumore puramente casuale, questa classe di attacchi adotta un approccio geometrico guidato volto a minimizzare la distanza statistica tra le classi. L'obiettivo fondamentale consiste nel manipolare selettivamente i vettori di input appartenenti alla classe di phishing al fine di proiettarli deliberatamente verso la porzione dello spazio iperdimensionale delle feature occupata stabilmente dai campioni della classe legittima. L'attaccante mira a scavalcare i confini decisionali precedentemente appresi dai modelli, forzando una classificazione erronea come falso negativo senza compromettere la funzionalità o l'efficacia del codice malevolo originale.

Motivazione

Nel dominio della sicurezza informatica e, nello specifico, del rilevamento del web phishing, l'introduzione del Mimicry Attack risponde alla necessità stringente di modellare uno scenario di minaccia proattivo e iper-realistico. Gli attaccanti del mondo reale non agiscono inserendo artefatti casuali o caotici all'interno del codice HTML o delle proprietà strutturali di una pagina clone. Al contrario, operano un'azione di ottimizzazione e mascheramento strategico: analizzano l'impronta digitale e le caratteristiche dei siti web legittimi più autorevoli e imitano chirurgicamente quelle precise metriche che i filtri anti-phishing monitorano come indicatori di sicurezza. Valutare la robustezza dei modelli di fronte a un avversario strategico consente di misurare non solo la stabilità matematica intrinseca degli algoritmi sotto perturbazioni generiche, bensì la loro reale resilienza e capacità di generalizzazione proattiva di fronte a tentativi intenzionali di evasione industriale.

Funzionamento

Il meccanismo matematico dell'attacco si basa sulla determinazione di un punto di riferimento altamente rappresentativo della classe legittima all'interno dello spazio delle feature, denominato centroide del profilo pulito. Al fine di rendere l'attacco immune all'influenza distorsiva di eventuali valori anomali (*outlier*) presenti nel traffico lecito, il centroide viene calcolato adottando la *mediana* multidimensionale dei campioni legittimi del set di addestramento, denotata come M_{leg} . Durante la fase di test, l'algoritmo di attacco intercetta esclusivamente i vettori originariamente etichettati come phishing (X_{phish}) e applica una traslazione lineare e coordinata lungo la direzione del centroide benigno. La perturbazione per un coefficiente di intensità dello shift $\alpha \in [0, 1]$ è governata dalla seguente equazione geometrica:

$$X_{\text{mimic}} = X_{\text{phish}} + \alpha \cdot (M_{\text{leg}} - X_{\text{phish}}) \quad (3.7)$$

Un valore di $\alpha = 0$ mappa la baseline originaria priva di alterazioni, mentre un valore di $\alpha = 1$ determina il collasso deterministico delle feature del sito malevolo sul valore mediano tipico dei siti legittimi, realizzando una mimetizzazione ideale lungo le dimensioni selezionate per l'offensiva.

Applicazione nel progetto

L'attacco prende di mira chirurgicamente le quattro feature strutturali e numeriche identificate come pilastri critici dallo studio di *Feature Occlusion*: `NoOfImage`, `LineOfCode`, `NoOfSelfRef` e `NoOfExternalRef`. Il profilo pulito di riferimento (M_{leg}) viene ricavato calcolando la mediana multidimensionale dei soli campioni legittimi all'interno del rispettivo set di addestramento. Lo shift lineare guidato dal coefficiente α viene testato sui livelli discreti dello 0%, 25%, 50%, 75% e 100%. Infine, i vettori modificati subiscono una procedura di sanificazione e ripristino dei vincoli fisici di dominio tramite arrotondamento ad interi positivi e *clipping*, garantendo la perfetta coerenza semantica dell'input prima della fase di inferenza. *Si nota tuttavia che tale attacco altera le feature in modo mutuamente indipendente; le limitazioni metodologiche derivanti da questa assunzione semplificativa sono discusse approfonditamente nella Sezione 4.3.2.*

Risultati ottenuti

Sotto il profilo delle metriche disaggregate si osserva un netto *effetto specchio*: l'accuratezza globale flette esclusivamente a causa del crollo della *Recall* del phishing e della simmetrica erosione della *Precision* del legittimo. Al contrario, la *Precision* della classe 0 e la *Recall* della classe 1 rimangono perfettamente

ancorate al valore ideale di 1.0, confermando che l'attacco agisce solo sui campioni malevoli senza introdurre falsi positivi.

Il confronto diretto rivela la netta superiorità della *Neural Network* rispetto al *Decision Tree* in termini di robustezza. Il *Decision Tree* mostra una degradazione “a gradini”: a un livello d'attacco iniziale ($\alpha = 25\%$), la *Recall* subisce un primo crollo, assestandosi fino ad $\alpha = 50\%$ (≈ 0.77), per poi registrare un secondo decremento a partire da $\alpha = 75\%$, stabilizzandosi a circa 0.67. Questo andamento riflette la natura rigida delle soglie ortogonali dell'albero: ulteriori traslazioni geometriche dei campioni non modificano la foglia di atterraggio finché non viene superata la soglia del nodo successivo, rendendo il modello temporaneamente immune all'incremento dell'intensità dell'attacco.

La *Neural Network* mostra un lievissimo calo. Il modello mostra un'immunità totale fino ad $\alpha = 25\%$ (1.0) e una flessione estremamente contenuta ad $\alpha = 50\%$ (≈ 0.98) e $\alpha = 75\%$ (≈ 0.97). Solo in condizioni di mimetizzazione totale ($\alpha = 100\%$), la *Recall* flette fino a circa 0.91. Anche al massimo livello di intensità dell'attacco, la *Neural Network* non converge con il *Decision Tree* ma preserva un ampio margine di sicurezza, dimostrando una superiorità strutturale definitiva nel contrasto ad attacchi avversari mirati. In questo caso, gli intervalli di confidenza al 95% risultano microscopici, conferendo un'elevata stabilità statistica alle curve di degradazione.

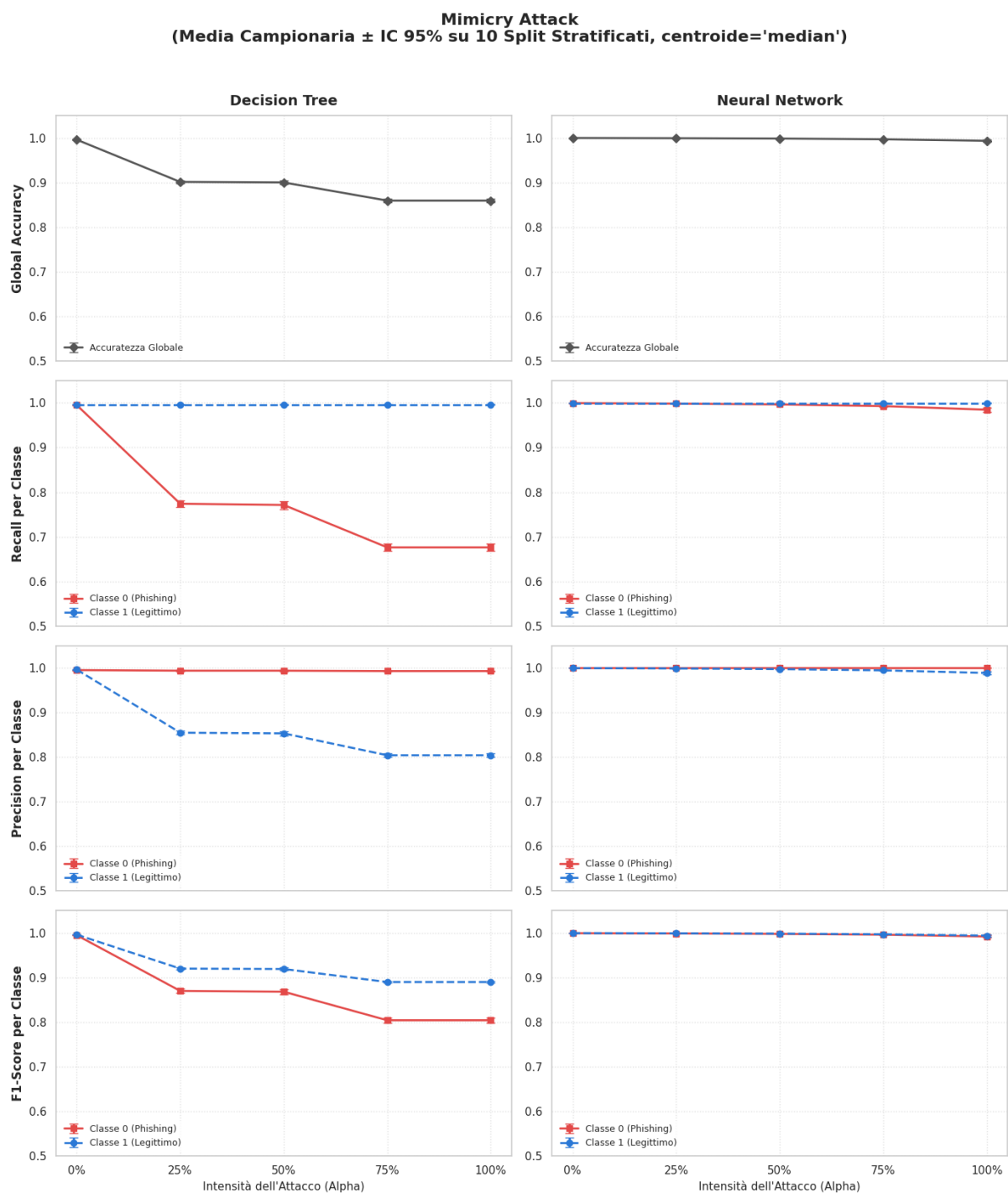


Figura 3.7: Degradazione performance Mimicry Attack: Decision Tree a sinistra, Neural Network a destra

Capitolo 4

Conclusioni

In questo progetto è stato analizzato il comportamento di due modelli in seguito a quattro tecniche di sporcamento del dataset.

4.1 Comparazione tra i modelli

Per le Neural Network, il calo delle prestazioni è sempre stato graduale in tutte e quattro le tecniche di sporcamento, mentre il Decision Tree ha mostrato un andamento simile solo nel caso del Missing Value, risultando invece più brusco e instabile nelle altre tecniche. Questo comportamento è causato dalla sua architettura basata su soglie rigide, che tende a collassare non appena un valore critico supera la soglia di split. A differenza della Neural Network dove l'informazione è distribuita su più neuroni. In scenari reali dove la qualità del dato non è garantita, la Neural Network risulterebbe quindi una scelta più affidabile rispetto al Decision Tree.

4.2 Efficacia metodi di sporcamento

Le quattro tecniche di corruzione applicate rispondono a scopi diversi.

Il **Missing Data (MCAR)** è utile per stimare le capacità dei modelli in condizioni realistiche di incompletezza dei dati. Dai risultati è emerso che la Neural Network è più robusta rispetto ad un Decision Tree in questo scenario realistico.

Il **Feature Occlusion Study**, più che una tecnica di sporcamento, ha svolto un ruolo investigativo: ha permesso di identificare quali feature sono più rappresentative di ciascun modello. Il suo valore nel progetto è quello di aver reso possibile la fase successiva di attacco mirato.

Il **Targeted Feature Noise** ha permesso di valutare i modelli nel caso in cui le feature più rilevanti per loro si degradino. Anche in questo caso, la Neural Network si è dimostrata il modello più robusto.

Il **Mimicry Attack** simula un attacco di evasione strategico a tempo di inferenza. Come nei restanti metodi, la Rete Neurale ha confermato una netta superiorità mentre il Decision Tree ha subito un collasso immediato a causa delle sue soglie decisionali rigide.

4.3 Limiti del progetto e sviluppi futuri

Nel corso del progetto sono emerse alcune limitazioni che è opportuno riconoscere esplicitamente, sia per contestualizzare correttamente i risultati ottenuti, sia come indicazione per eventuali sviluppi futuri.

4.3.1 Specificità del dataset

Il dataset PhiUSIIL è stato costruito per uno studio accademico specifico, con feature calcolate secondo metodologie e parametri di estrazione molto precisi. Questo garantisce dati di alta qualità, privi di valori mancanti e con una struttura ben definita. Però, allo stesso tempo, limita la generalizzabilità dei

risultati. Un dataset raccolto in un contesto reale potrebbe avere feature calcolate con criteri diversi, distribuzioni differenti o un rumore presente nei dati prima ancora di applicare lo sporcamento artificiale. Le conclusioni sulla robustezza dei modelli, per quanto solide sui dati analizzati, andrebbero quindi validate anche su dataset diversi prima di poter essere generalizzate. Come sviluppo futuro, sarebbe utile ripetere l'analisi su altri dataset raccolti in contesti diversi, per verificare se la robustezza dei modelli produce risultati simili.

4.3.2 Indipendenza tra le feature e vincoli strutturali

Le tecniche di sporcamento applicate in questo studio trattano ogni feature come statisticamente indipendente, perturbandola singolarmente. Questa assunzione è in contrasto con quanto emerso dall'analisi di correlazione (Sezione 2.1.2). Ignorare tali interdipendenze comporta il rischio di generare combinazioni di valori irrealistiche e semanticamente incoerenti rispetto alla reale natura di un URL o di una pagina web. Questa limitazione emerge in modo critico nel *Mimicry Attack*. L'algoritmo implementato ha modificato solo quattro variabili: `NoOfImage`, `LineOfCode`, `NoOfSelfRef` e `NoOfExternalRef`, tramite il coefficiente di intensità α , mantenendo però inalterate metriche ad esse strettamente correlate, come ad esempio `NoOfJS` o `NoOfCSS`.

Questo approccio solleva due importanti criticità metodologiche:

1. **Generazione di istanze anomali:** Lo shift geometrico isolato può proiettare vettori verso valori così sbilanciati che risulterebbero facilmente intercettabili da un controllo di consistenza come anomalie statistiche improbabili in un vero sito web.
2. **Sottostima dell'efficacia dell'attacco:** Come evidenziato nella Sezione 3.4, la Neural Network ha mostrato un'eccezionale resilienza proprio perché capace di estrarre il segnale discriminante dalle restanti 37 feature rimaste intatte. In uno scenario reale, un attaccante modificherebbe l'intero blocco di variabili co-dipendenti per non compromettere la funzionalità della pagina, privando il modello anche di quelle dimensioni che ne hanno garantito l'elevata *Recall*.

Come cruciale sviluppo futuro, si prospetta l'evoluzione verso un *Constrained Mimicry Attack* capace di preservare i reali vincoli del dominio. Tale estensione potrà basarsi sulla matrice di correlazione di Spearman per propagare linearmente la perturbazione alle feature co-dipendenti, ad esempio tramite la decomposizione di Cholesky.

4.3.3 T student nel Repeated Hold-Out

La validazione sperimentale delle tecniche di sporcamento è stata condotta adottando una strategia di Repeated Stratified Hold-Out basata su $N = 10$ ripetizioni. Un numero superiore di iterazioni, come ad esempio $N = 30$, avrebbe garantito una stabilità asintotica ancora maggiore delle metriche e degli intervalli di confidenza, riducendo la sensibilità dei risultati alla specifica sequenza di partizionamento casuale generata. La scelta di arrestare il processo a 10 split rappresenta un compromesso ottimale rispetto al costo computazionale complessivo. Ogni singola ripetizione richiede infatti il riaddestramento completo sia del Decision Tree che della Neural Network. Tale lavoro si moltiplica in modo critico durante la fase di *Feature Occlusion*, in cui la procedura di inferenza deve essere ripetuta sistematicamente per ciascuna delle 41 feature del dataset. Di conseguenza, un ulteriore incremento delle repliche avrebbe comportato tempi di esecuzione non compatibili con le risorse di calcolo e le tempistiche disponibili per lo sviluppo del progetto.

Dal punto di vista statistico, va sottolineato che gli split ottenuti con il repeated hold-out non costituiscono osservazioni indipendenti tra loro. Questo è dovuto alla parziale e inevitabile sovrapposizione delle istanze di addestramento tra una ripetizione e l'altra. Se si applica un t-test appaiato standard, senza considerare questa correlazione, si rischia di sottostimare la varianza reale della stima delle partizioni. Questo rende il test troppo liberale, aumentando il rischio di considerare significative delle differenze che rientrano nella normale variabilità campionaria [3].

Abbiamo scelto di mantenere questo approccio per preservare la coerenza con il resto delle analisi del progetto. Come sviluppo futuro, si potrebbe integrare uno stimatore di varianza corretto per il repeated hold-out, come la formulazione di Nadeau-Bengio [4], al fine di correggere l'effetto dell'indipendenza mancata dei campioni e ottenere una stima più conservativa e affidabile della significatività statistica.

Bibliografia

- [1] Haldun Akoglu. “User’s guide to correlation coefficients”. In: *Turkish Journal of Emergency Medicine* 18.3 (2018), pp. 91–93.
- [2] Roderick JA Little e Donald B Rubin. *Statistical analysis with missing data*. John Wiley & Sons, 2019.
- [3] Thomas G Dietterich. “Approximate statistical tests for comparing supervised classification learning algorithms”. In: *Neural computation* 10.7 (1998), pp. 1895–1923.
- [4] Claude Nadeau e Yoshua Bengio. “Inference for the generalization error”. In: *Advances in neural information processing systems* 12 (1999).